AstroBurst

High-Performance Astronomical Image Processor

Technical Document — Release v0.4.5

Samuel Krieger Bonini

March 2026

Field	Value
Document Version	0.4.5
Release	v0.4.5
Date	March 2026
Author	Samuel Krieger Bonini
Stack	Rust 1.75+ · Tauri v2.10 · React 19 · TypeScript 5.7
Platforms	Windows · macOS · Linux
License	GPL-3.0-or-later
Repository	https://github.com/samuelkriegerbonini-dev/AstroBurst

Contents

1 Executive Summary **3**

1.1 What's New in v0.4.5 . . . 3

1.2 What's New in v0.4.2 . . . 3

1.3 What's New in v0.4.0 . . . 3

2 System Architecture **4**

2.1 Backend (Rust) . . . 4

2.2 Non-Destructive Composite Pipeline (v0.4.5) . . . 5

2.3 Frontend (React + TypeScript) . . . 5

2.4 IPC Strategy . . . 6

3 Mathematical Foundations **6**

3.1 Screen Transfer Function (STF) . . . 6

3.2 Asinh Stretch . . . 7

3.3 Masked Stretch (v0.4.2) . . . 7

3.4 Cross-Correlation Alignment (Phase Correlation) 7
3.5 Star-Based Affine Alignment (v0.4.0) 7
3.6 Sigma-Clipped Stacking 8
3.7 Channel Blending (v0.4.2) 8
3.8 White Balance (v0.4.0, non-destructive v0.4.5) 8
3.9 SCNR (v0.4.0, non-destructive v0.4.5) 8
3.10 Drizzle Integration 9
3.11 Richardson-Lucy Deconvolution 9
3.12 Background Extraction 9
3.13 Wavelet Denoising 9
3.14 Calibration Pipeline 9
3.15 Star Detection 9
3.16 SPCC (Spectrophotometric Color Calibration, v0.4.2) 10

4 Command Module Reference **10**
4.1 Compose Module (`cmd/compose.rs`) 10
4.2 Image, Analysis, Processing, Stacking, Cube, Astrometry, Config 10

5 FITS and ASDF I/O **10**

6 WebGPU Compute Pipeline **11**

7 Performance Benchmarks **11**

8 Dependencies **11**

9 Known Limitations **11**

10 Roadmap **12**

A Sample Test Data **12**

1 Executive Summary

AstroBurst is a cross-platform desktop application for processing, analyzing, and visualizing astronomical FITS and ASDF images. It is the first astronomical image processor built on the **Rust/Tauri/WebGPU** stack, combining native-level performance with a modern, accessible user interface.

The application targets two user segments:

- Professional astronomers working with observatory data (JWST, Hubble, Roman Space Telescope, ground-based IFU instruments)
- Advanced astrophotographers processing deep-sky narrowband and broadband imaging data

Version 0.4.5 delivers a complete processing pipeline from raw FITS/ASDF ingestion through calibration, stacking, alignment, color composition, and export, with **47 backend commands** exposed through a typed TypeScript API layer and a 10-step ComposeWizard pipeline.

1.1 What's New in v0.4.5

- **Non-destructive composite pipeline** — Immutable ORIG cache with idempotent WB and SCNR; `reset_wb_cmd` for one-click undo
- **STF rendering consistency** — GPU shader, CPU worker, and Rust backend produce pixel-identical output via symmetric denominator protection and unified padding threshold
- **Blend preset positional fallback** — SHO/HOO/Foraxx presets work with any bin configuration via positional weight mapping
- **Spectroscopy wavelength unit conversion** — Automatic CUNIT3 detection (M, NM, ANGSTROM, HZ) with display conversion
- **Export pipeline fix** — Exports use user STF params instead of identity, matching preview

1.2 What's New in v0.4.2

- **ComposeWizard** — 10-step pipeline (Channels, Stack, BG, Align, Blend, Color, Mask, Stretch, Adjust, Export)
- **Synthetic FITS generation** — Star field, PSF, and noise injection for testing
- **Narrowband palette system** — SHO, HOO, HOS, NaturalColor, Custom with auto-detection
- **Live composite STF re-stretch** — Per-channel shadow/midtone/highlight without re-composing
- **SPCC** — Spectrophotometric color calibration with optional Gaia DR3 TAP (vizier feature)
- **Masked stretch** — Iterative MTF with star mask protection
- **PSF enhancement** — Moment-based FWHM with subpixel peak estimation

1.3 What's New in v0.4.0

- **Star-based affine alignment** — Triangle asterism matching with RANSAC, automatic fallback to FFT phase correlation
- **Stability-based auto white balance** — Lowest MAD/median channel as reference
- **SCNR luminance redistribution** — ITU-R BT.709 weighted redistribution to R and B

- **Numerical correctness audit** — NaN ordering, Hann window, polynomial basis, RL regularization, phase correlation confidence

2 System Architecture

AstroBurst follows a strict backend/frontend separation, with all computation offloaded to Rust and all rendering handled by the browser engine embedded in Tauri.

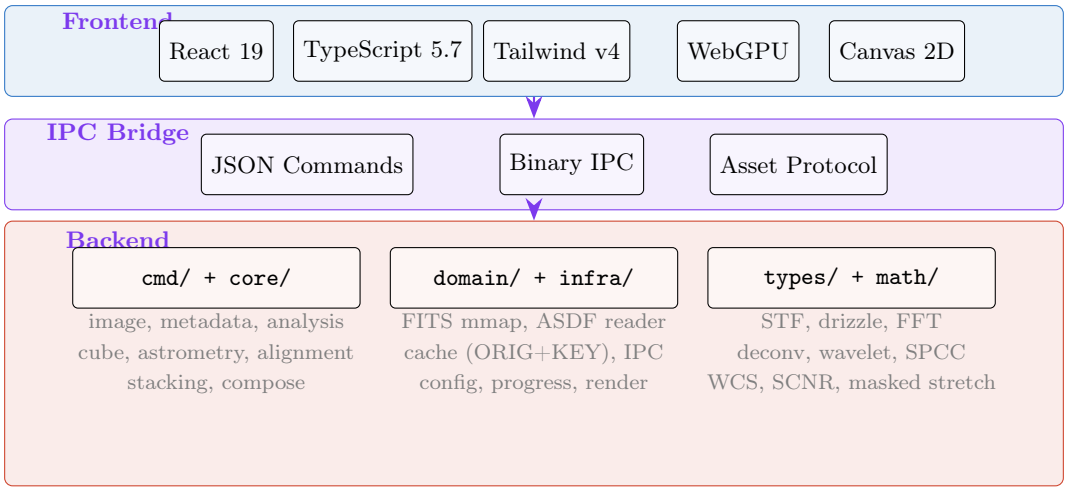

Figure 1: AstroBurst three-layer architecture

2.1 Backend (Rust)

The Rust backend is organized into six layers:

`cmd/` Tauri command handlers organized by domain (47 registered commands). Each module receives deserialized JSON arguments, delegates to core/domain logic, and returns JSON or binary responses.

`core/` Pure computational modules: alignment (phase correlation, affine with triangle asterism matching), analysis (histogram, FFT, stars), astrometry (plate solve, WCS, SPCC), compose (RGB, channel blend, SCNR, white balance, LRGB), cube (spectrum, frames), imaging (STF, deconvolution, wavelet, background, masked stretch, PSF estimation, star mask), metadata (header discovery, filter detection), stacking (sigma-clip, drizzle), synth (star field, PSF, noise).

`domain/` Orchestration layer coordinating core modules for complex multi-step operations (calibration masters, drizzle-from-paths, cube processing, plate solve).

`infra/` Infrastructure: Arc-based zero-copy cache with ORIG/KEY dual-layer for non-destructive composite editing, config management, FITS I/O (mmap parser, MEF scanner, FITS writer with multi-BITPIX), ASDF reader (YAML tree parser, binary block extractor, zlib/bzip2/lz4 decompressor, Roman data model converter, gWCS bridge), binary IPC encoding, progress tracking, PNG rendering (8/16-bit grayscale and RGB).

`types/` Shared type definitions: image stats, STF params, stacking/drizzle/RL configs, compose types (AlignMethod, WhiteBalance, BlendWeight), SCNR, error types, FITS header abstraction, 310+ constants.

`math/` Mathematical primitives: NaN-safe median/MAD, sigma clipping, SIMD log approximation (Cephes degree-8 minimax, < 1 ULP precision).

2.2 Non-Destructive Composite Pipeline (v0.4.5)

The composite cache uses a dual-layer architecture:

ORIG/KEY Cache Architecture

Layer	Keys	Semantics
ORIG (immutable)	`__composite_orig_r/g/b`	Written once by blend/compose/RGB-FITS load
KEY (working)	`__composite_r/g/b`	Reconstructed: ORIG $\times$ WB $\rightarrow$ KEY, or ORIG $\times$ WB $\rightarrow$

Every downstream operation (`calibrate_composite_cmd`, `apply_scnr_cmd`) reads from ORIG, applies the full transform chain, and writes to KEY. This makes WB and SCNR idempotent: applying the same factors twice produces identical results. `reset_wb_cmd` copies ORIG $\rightarrow$ KEY.

2.3 Frontend (React + TypeScript)

The frontend is organized around the ComposeWizard (10-step pipeline) and split preview contexts:

`services/` 12 domain-specific service modules wrapping Tauri IPC: compose, fits, analysis, header, cube, stacking, processing, config, astrometry, export, synth, tiles.

`ComposeWizard`
: 10-step pipeline: Channels $\rightarrow$ Stack $\rightarrow$ BG $\rightarrow$ Align $\rightarrow$ Blend $\rightarrow$ Color $\rightarrow$ Mask $\rightarrow$ Stretch $\rightarrow$ Adjust $\rightarrow$ Export. Internal `useReducer` state with step completion tracking and suggested-next-step logic.

Preview Contexts
: 8 split React contexts (FileCtx, DoneFilesCtx, HistCtx, CubeCtx, RgbCtx, RenderCtx, RawPixelsCtx, NarrowbandCtx) for granular re-render control.

GPU Rendering
: WebGPU singleton with WGSL fragment shader for real-time STF. CPU worker fallback with identical MTF implementation. Both use symmetric denominator protection and padding threshold matching the Rust backend.

2.4 IPC Strategy

Binary IPC Protocol

The `get_raw_pixels_preview` command encodes a **16-byte header** followed by raw pixel data:

Offset	Type	Size	Description
0	`u32`	4B	Image width
4	`u32`	4B	Image height
8	`f32`	4B	Data minimum
12	`f32`	4B	Data maximum
16	`f32[]`	$W \times H \times 4$B	Raw pixel values

3 Mathematical Foundations

3.1 Screen Transfer Function (STF)

The STF implements the PixInsight-style **Midtone Transfer Function** (MTF). Given a normalized pixel value $x \in [0, 1]$ and midtone parameter $m \in (0, 1)$:

$$\boxed{\mathrm{MTF}(x, m) = \frac{(m-1) \cdot x}{(2m-1) \cdot x - m}} \tag{1}$$

Properties:

- $\mathrm{MTF}(0, m) = 0$ and $\mathrm{MTF}(1, m) = 1$ for all m
- $\mathrm{MTF}(m, m) = 0.5$ — the midtone maps to 50% brightness
- $m < 0.5$ brightens the image; $m > 0.5$ darkens it
- When $m = 0.5$: $\mathrm{MTF}(x, 0.5) = x$ (identity)

Denominator Protection (v0.4.5)

The denominator $D = (2m-1)x - m$ changes sign across the valid domain. The correct guard is:

$$\text{if } |D| < \varepsilon \text{ then return } x; \quad \text{if } |m - 0.5| < \varepsilon \text{ then return } x$$

The v0.4.2 GPU shader incorrectly used $\max(D, \varepsilon)$ which forces $D > 0$, inverting the transfer function for $m < 0.5$. v0.4.5 uses symmetric $|D|$ protection across GPU (WGSL), CPU worker (JS), and Rust backend, achieving pixel-identical output.

Padding pixels ($v \leq 10^{-7}$) are rendered as black across all three rendering paths, matching the Rust `is_valid_pixel` threshold.

The **auto-stretch** computes parameters from image statistics:

$$\text{shadow} = \max\left(0,\ \tilde{x}_{\text{norm}} + k \cdot \sigma_{\text{norm}}\right), \quad k = -2.8 \tag{2}$$

$$\text{midtone} = \mathrm{MTF}^{-1}_{\text{balance}}\left(\frac{\tilde{x}_{\text{norm}} - \text{shadow}}{1 - \text{shadow}},\ t_{\text{bg}}\right), \quad t_{\text{bg}} = 0.25 \tag{3}$$

where $\tilde{x}_{\text{norm}}$ and σ_{norm} are the normalized median and sigma (MAD $\times$ 1.4826).

3.2 Asinh Stretch

$$\boxed{f(x) = \frac{\operatorname{asinh}(\alpha \cdot x)}{\operatorname{asinh}(\alpha)}} \tag{4}$$

Maps $[0, 1] \to [0, 1]$, approximately linear for small x (preserving faint detail), logarithmic compression for bright values. $\alpha \in [1, 500]$ controls aggressiveness.

3.3 Masked Stretch (v0.4.2)

Iterative MTF stretch with star mask protection. At each iteration k:

$$M^{(k)} = \text{StarMask}(I^{(k)},\ \text{growth},\ \text{softness}) \tag{5}$$

$$I^{(k+1)} = (1 - \alpha \cdot M^{(k)}) \cdot \text{MTF}(I^{(k)}) + \alpha \cdot M^{(k)} \cdot I^{(k)} \tag{6}$$

where α is the protection amount (default 0.85) and growth controls the mask dilation radius. Convergence when background level reaches target (default 0.25). JWST-optimized defaults: growth 0.8, protection 96%.

3.4 Cross-Correlation Alignment (Phase Correlation)

$$\text{shift} = \arg \max_{(\Delta x, \Delta y)} \mathcal{F}^{-1} \left\{ \frac{\hat{R} \cdot \hat{T}^*}{|\hat{R} \cdot \hat{T}^*|} \right\} \tag{7}$$

Hann window: $w(n) = 0.5(1 - \cos(2\pi n/N))$. Confidence: $z = (P_{\text{peak}} - \mu_P)/\sigma_P$. Coarse-to-fine: downsample to 512×512, refine on centered crop.

3.5 Star-Based Affine Alignment (v0.4.0)

Triangle descriptor (scale-invariant):

$$T = \left(\frac{d_{\text{mid}}}{d_{\text{short}}}, \frac{d_{\text{long}}}{d_{\text{short}}} \right), \quad \text{match if } |T_r - T_t| < 0.005 \tag{8}$$

RANSAC (500 iterations) fits 6-DOF affine or 4-DOF rigid:

$$\begin{pmatrix} x_t \\ y_t \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x_r \\ y_r \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix} \tag{9}$$

Sanity: offset $< 25\%$, rotation $< 5°$, scale $\in [0.85, 1.15]$, residual < 3 px, inlier ratio $> 30\%$. Fallback: affine $\to$ rigid $\to$ phase correlation $\to$ identity. Warp via bicubic Catmull-Rom, parallelized per row.

3.6 Sigma-Clipped Stacking

Algorithm 1: Sigma-Clipped Stacking (median+MAD first iter, mean+std subsequent)

Frames $\{F_1, \ldots, F_N\}$, $\sigma_{\text{low}}, \sigma_{\text{high}}$, max iters K Stacked image S **foreach** *pixel* (i, j) **do**
  $V \leftarrow \{F_k(i, j)\}$, iter 0: $\mu = \text{median}(V)$, $\sigma = \text{MAD}(V) \times 1.4826$;
  iters 1+: $\mu = \text{mean}(V)$, $\sigma = \text{std}(V)$;
  reject $v \notin [\mu - \sigma_{\text{low}} \cdot \sigma,\ \mu + \sigma_{\text{high}} \cdot \sigma]$;
  $S(i, j) \leftarrow \text{mean}(V)$;
end

3.7 Channel Blending (v0.4.2)

Given N input channels C_k and weight matrix W:

$$\text{Out}_c = \sum_{k=1}^{N} W_{k,c} \cdot C_k, \quad c \in \{R, G, B\} \tag{10}$$

Preset weights (e.g., SHO: SII $\rightarrow R$, H$\alpha \rightarrow G$, OIII $\rightarrow B$) use named channelId matching with **positional fallback** (v0.4.5): when preset channelIds don't match filled bins, weights map positionally to the first N filled bins.

3.8 White Balance (v0.4.0, non-destructive v0.4.5)

Reference channel selection by minimum coefficient of variation:

$$k^* = \arg\min_{k \in \{R,G,B\}} \frac{\text{MAD}_k}{\tilde{C}_k}; \qquad w_j = \frac{\tilde{C}_{k^*}}{\tilde{C}_j},\ j \neq k^*; \quad w_{k^*} = 1 \tag{11}$$

In v0.4.5, WB is applied from ORIG cache: $\text{KEY}_c = \text{ORIG}_c \times w_c$. Idempotent.

3.9 SCNR (v0.4.0, non-destructive v0.4.5)

Two limit methods and optional luminance preservation:

$$G' = \min\left(G,\ \tfrac{R+B}{2}\right) \quad \text{or} \quad \min(G,\ \max(R, B)) \tag{12}$$

$$G_{\text{new}} = G + \alpha(G' - G), \quad \alpha \in [0, 1] \tag{13}$$

When luminance preservation is enabled (BT.709):

$$\Delta = \frac{0.7152 \cdot (G - G_{\text{new}})}{0.2126 + 0.0722}; \quad R' = \min(R + \Delta, 1); \quad B' = \min(B + \Delta, 1) \tag{14}$$

In v0.4.5, SCNR is applied from ORIG cache with WB pre-applied: ORIG $\times$ WB $\rightarrow$ SCNR $\rightarrow$ KEY. Idempotent.

3.10 Drizzle Integration

$$O(i,j) = \frac{\sum_p w_p \cdot K(i - x_f^p,\ j - y_f^p) \cdot I_p}{\sum_p w_p \cdot K(i - x_f^p,\ j - y_f^p)} \tag{15}$$

Three kernels: square (top-hat), Gaussian, Lanczos-3. Flat-storage accumulator (v0.4.0): `Vec<f32>` + `Vec<u16>` replacing `Vec<Vec<f32>>`. Finalize uses median/MAD sigma clipping (Stetson 1987).

3.11 Richardson-Lucy Deconvolution

$$f^{(k+1)} = f^{(k)} \cdot \left(\frac{g}{f^{(k)} * h} * \hat{h} \right) \cdot \frac{1}{1+\lambda} \tag{16}$$

Tikhonov regularization as multiplicative damping ($\lambda = 0.001$ default). FFT-based convolution.

3.12 Background Extraction

$$B(x,y) = \sum_{i=0}^{d} \sum_{j=0}^{d-i} a_{ij}\, x^i\, y^j \tag{17}$$

Polynomial degree 1–5, grid 3–32, sigma-clipped sampling with `powi` exponentiation.

3.13 Wavelet Denoising

A trous wavelet decomposition: $I = \sum_{k=1}^{N} W_k + c_N$. Hard threshold: $W'_k = W_k$ if $|W_k| > \sigma_k t_k$, else 0. Up to 8 scales.

3.14 Calibration Pipeline

$$\boxed{I_{\text{cal}} = \frac{I_{\text{raw}} - M_{\text{bias}} - r \cdot M_{\text{dark}}}{M_{\text{flat}} / \bar{M}_{\text{flat}}}} \tag{18}$$

Masters via median combine. Flat normalized by mean. Negative pixels clamped to zero.

3.15 Star Detection

Tile-based background (parallelized). Connected-component analysis with flux-weighted centroid:

$$\bar{x}_C = \frac{\sum_{(i,j)\in C} i \cdot (I_{ij} - b)}{\sum_{(i,j)\in C} (I_{ij} - b)} \tag{19}$$

FWHM from second intensity moments (true **median** across detected stars, v0.4.5):

$$\text{FWHM} = 2\sqrt{2\ln 2} \cdot \sqrt{\frac{\sigma_x^2 + \sigma_y^2}{2}} \tag{20}$$

3.16 SPCC (Spectrophotometric Color Calibration, v0.4.2)

Color correction factors derived from stellar Bp-Rp color indices matched against a white reference spectrum. Optional Gaia DR3 TAP query (`vizier` feature flag) for real catalog data; built-in Bp-Rp estimation fallback.

4 Command Module Reference

The backend exposes **47 Tauri commands** across 12 modules.

4.1 Compose Module (`cmd/compose.rs`)

Command	Parameters	Returns
`compose_rgb_cmd`	r/g/b_path, stretch/wb/scnr, align_method	png, STF, offsets
`blend_channels_cmd`	channel_paths[], weights[], preset	png, STF, lum FITS
`align_channels_cmd`	paths[], align_method	per-channel aligned FITS
`restretch_composite_cmd`	per-channel STF, SCNR opts	png
`calibrate_composite_cmd`	r/g/b_factor	png (ORIG × WB)
`apply_scnr_cmd`	method, amount, r/g/b_factor	png (ORIG × WB × SCNR)
`compute_auto_wb_cmd`	—	r/g/b factors, ref channel
`reset_wb_cmd`	output_dir	png (ORIG → KEY)
`clear_composite_cache_cmd`	—	—
`update_composite_channel_cmd`	channel, path	updated flag
`export_aligned_channels_cmd`	r/g/b_path, align, WCS opts	per-channel FITS

Table 1: Compose module (11 commands). All WB/SCNR commands read from ORIG cache.

4.2 Image, Analysis, Processing, Stacking, Cube, Astrometry, Config

Remaining 36 commands cover: `process_fits`, `process_fits_full`, `get_raw_pixels_preview`, `export_fits`, `export_fits_rgb`, `export_png`, `export_rgb_png`, `resample_fits_cmd`, `compute_histogram`, `compute_fft_spectrum`, `detect_stars`, `apply_stf_render`, `generate_tiles`, `deconvolve_rl_cmd`, `extract_background_cmd`, `wavelet_denoise_cmd`, `masked_stretch_cmd`, `apply_arcsinh_stretch_cmd`, `estimate_psf_cmd`, `spcc_calibrate_cmd`, `calibrate`, `stack`, `drizzle_stack_cmd`, `drizzle_rgb_cmd`, `run_pipeline_cmd`, `process_cube_cmd`, `process_cube_lazy_cmd`, `get_cube_info`, `get_cube_frame`, `get_cube_spectrum`, `plate_solve_cmd`, `get_wcs_info`, `detect_narrowband_filters`, `get_config`, `update_config`, `save_api_key`, `get_api_key`, `generate_synth_cmd`, `generate_synth_stack_cmd`.

5 FITS and ASDF I/O

FITS via `memmap2` with BSCALE/BZERO: $v_{\text{phys}} = \text{BSCALE} \times v_{\text{stored}} + \text{BZERO}$. Multi-BITPIX export: 16 (auto BZERO/BSCALE), −32 (float32), −64 (float64).

ASDF: first Rust implementation. YAML tree → NdArrayMeta → binary block (zlib/bzip2/lz4) → Array2. Roman data model traversal. Auto-dispatch by extension.

6 WebGPU Compute Pipeline

WGSL fragment shader implementing MTF with symmetric denominator guard. `r32float` texture, 6-vertex fullscreen quad. Uniform buffer: [dataMin, dataMax, shadow, midtone, highlight, W, H, pad] (32 bytes). CPU worker fallback with identical `stfTransfer` function.

7 Performance Benchmarks

Operation	Input Size	Time	Notes
Single FITS processing	4096×4096	120 ms	533 MB/s
Batch (10 frames)	10×64 MB	450 ms	1.4 GB/s (Rayon)
Histogram + auto-STF	4096×4096	35 ms	NaN-safe
Star detection (σ=5)	4096×4096	80 ms	∼3000 stars
Sigma-clip stack	10×64 MB	2.1 s	5 iters
Drizzle 2×	10×64 MB	4.2 s	Flat accumulator
Channel blend (3ch)	4096×4096 × 3	0.4 s	+ auto STF + lum FITS
Affine alignment	4096×4096	0.8 s	80 stars, RANSAC
Masked stretch (10 iter)	4096×4096	1.2 s	Star mask + MTF
WebGPU STF render	4096×4096	8 ms	GPU compute
WB (non-destructive)	4096×4096 × 3	45 ms	ORIG read + factor

Table 2: Benchmarks (AMD Ryzen 9 7950X, 64 GB DDR5, NVMe). Binary: ∼15 MB.

8 Dependencies

Crate	Version	Purpose
`tauri`	2.10	Application framework, IPC, asset protocol
`ndarray`	0.16	N-dimensional arrays, Rayon parallel iterators
`rayon`	1.10	Data-parallel batch processing
`memmap2`	0.9	Memory-mapped FITS I/O
`rustfft`	6.4	FFT for deconvolution, phase correlation, power spectrum
`image`	0.25	PNG encoding (8/16-bit)
`serde_yaml`	0.9	ASDF YAML tree parsing
`flate2`	1.0	zlib decompression (ASDF blocks)
`bzip2`	0.5	bzip2 decompression (ASDF, default feature)
`lz4_flex`	0.11	lz4 decompression (ASDF, default feature)
`reqwest`	0.12	Astrometry.net API, Vizier TAP (optional)

9 Known Limitations

1. No non-linear distortion correction (barrel/pincushion)
2. Online plate solving only (local index planned)
3. WebGPU Chromium 113+ required; Canvas 2D fallback available
4. No session persistence
5. ASDF read-only (writer not implemented)
6. No MAST API integration (manual download required)
7. Cube "Collapse Sum" is actually collapse mean (labeled correctly in v0.4.5)

10 Roadmap

Feature	Priority	Status
Star-based affine alignment	High	Done (v0.4.0)
Non-destructive WB/SCNR	High	Done (v0.4.5)
ComposeWizard pipeline	High	Done (v0.4.2)
Masked stretch	High	Done (v0.4.2)
SPCC calibration	High	Done (v0.4.2)
STF GPU/CPU consistency	High	Done (v0.4.5)
Multi-extension FITS (ERR/DQ/VAR)	High	Planned (v0.5)
MAST API integration	High	Planned (v0.5)
WCS overlay on preview	Medium	Planned (v0.5)
Batch pipeline	Medium	Planned (v0.5)
Star removal	Medium	Planned (v0.6)
Photometric calibration (Gaia DR3)	Medium	Planned (v0.6)
Full GPU pipeline (WebGPU compute)	Medium	Planned (v1.0)

A Sample Test Data

HST/WFPC2 narrowband FITS in `tests/sample-data/`: 502nmos.fits ([OIII] 502 nm, Blue), 656nmos.fits (Hα 656 nm, Green), 673nmos.fits ([SII] 673 nm, Red). 1600×1600 float32, WCS TAN. Public domain (NASA/ESA).